FilaFácil Logística

Python + SQLAlchemy explicado para quem vem do Laravel

Cada conceito do projeto é comparado com o que você já conhece de Laravel, Eloquent, Artisan e Composer.



Legenda de cores usada em todo o guia: vermelho = lado Laravel, azul = lado Python.

1. A ideia central

Imagine um app Laravel com um model **Pickup**, uma tabela **pickups**, um **PickupController**, um **PickupService** e um **PickupRepository**. Agora troque duas coisas: em vez de rotas HTTP e views Blade, a interface é um **menu no terminal**; e em vez de MySQL, o banco é um **arquivo SQLite** (filafacil.db). É exatamente isso que este projeto é.

A arquitetura em 3 camadas é a mesma que você usa quando aplica Service e Repository no Laravel. A diferença é que no Laravel isso é uma convenção da comunidade; aqui foi uma regra rígida do projeto: **cada camada só conversa com a de baixo, e só o repository toca no banco**.

1.1 Mapa de pastas

Laravel / PHP	Este projeto / Python
app/Http/Controllers/PickupController.php	`controller/pickup_controller.py`
app/Services/PickupService.php	`service/pickup_service.py`
app/Repositories/PickupRepository.php	`repository/pickup_repository.py`
app/Models/Pickup.php	`models/pickup.py`
config/database.php + .env (conexão)	`database.py` (DATABASE_URL, engine)
database/migrations/ (cria a tabela)	`database.py`: função `create_tables()`
routes/ + artisan (ponto de entrada)	`main.py`
composer.json	`requirements.txt`
vendor/	`.venv/` (ambiente virtual)
php no composer.json / .php-version	`.python-version` (fixa Python 3.11)
database.sqlite (se usasse SQLite)	`filafacil.db` (criado sozinho, fora do Git)

> **Resumo da analogia**
> O **controller** deste projeto se parece mais com um **Artisan Command interativo** (`$this->ask()`, `$this->choice()`, `$this->table()`) do que com um controller HTTP. Guarde essa imagem, ela ajuda muito na seção 5.

2. O caminho de uma operação

Uma ação desce do menu até o banco e a resposta volta pelo mesmo caminho. Nenhuma camada pula a seguinte.

Request + Route	Controller	Service	Repository	Eloquent / Query Builder	MySQL
Usuário digita	Controller	Service	Repository	Session (SQLAlchemy)	SQLite filafacil.db

2.1 Exemplo: "Cadastrar coleta" passo a passo

#	Quem	O que acontece	Equivalente Laravel
1	Controller	Chama `input()` para ler nome, endereço, pacotes e prioridade. Converte `int()` e trata erro.	FormRequest lendo o Request / `$this->ask()`
2	Controller	Chama `service.register(...)` e captura `ValueError`.	Controller chamando o Service e tratando ValidationException
3	Service	Faz `strip()`, rejeita vazio, pacotes <= 0, valores fora de PRIORITIES/STATUSES.	Regras do FormRequest / Validator
4	Service	Só então chama `repository.create(...)`.	Service chamando o Repository
5	Repository	Abre `with SessionLocal() as session:`, faz `add`, `commit`, `refresh`.	`Pickup::create([...])`
6	SQLAlchemy	Gera o `INSERT INTO pickups ...` e envia ao SQLite.	Eloquent gerando o INSERT
7	Controller	Recebe o objeto com `id` preenchido e imprime "Coleta #id cadastrada".	Retornar a resposta / `$this->info()`

Por que três camadas?
Se amanhã o banco mudar (SQLite para PostgreSQL) só o **repository** e o **database.py** mudam. Se o menu virar uma API, só o **controller** muda. As regras de negócio (**service**) ficam intactas. É o mesmo argumento que justifica Service/Repository no Laravel.

3. Python para quem vem de PHP

Antes dos arquivos do projeto, um dicionário de sintaxe. Quase tudo que você faz em PHP tem um equivalente direto.

Laravel / PHP	Este projeto / Python
`$name = "Ana";`	`name = "Ana"` (sem `$`, sem `;`)
Blocos com `{ }`	Blocos por **indentação** (4 espaços) depois de `:`. A indentação faz parte da sintaxe.
`function sum(int $a): int`	`def sum(a: int) -> int:`
`class A { function __construct() }`	`class A:` com `def __init__(self):`
`$this->repository`	`self.repository` (`self` é declarado explicitamente em todo método)
`null`	`None` (compare com `is None`)
`['a' => 1]` e `[1, 2]` (array para tudo)	`{'a': 1}` é dict, `[1, 2]` é list, `(1, 2)` é tuple (imutável)
`"Olá {$name}"`	`f"Olá {name}"` (f-string)
`foreach ($items as $i)`	`for i in items:`
`array_map(fn($x) => $x*2, $a)`	`[x * 2 for x in a]` (list comprehension)
`throw new InvalidArgumentException()`	`raise ValueError("mensagem")`
`try { } catch (E $e) { }`	`try:` ... `except ValueError as error:`
`use App\Models\Pickup;`	`from models.pickup import Pickup`
`public / private`	Não existe imposição. Convenção: nome começando com `_` significa "uso interno".
`__toString()`	`__repr__()` (texto legível do objeto, usado em `print` e no debug)
`/** PHPDoc */`	`"""docstring"""` logo abaixo de `def` ou `class`
camelCase em métodos, PascalCase em classes	**snake_case** em funções, variáveis, arquivos e colunas; PascalCase só em classes; MAIÚSCULAS em constantes
`try { } finally { close(); }`	`with SessionLocal() as session:` fecha a sessão sozinho no fim do bloco (context manager)

Atenção: type hints não são validados
Em PHP, `declare(strict_types=1)` faz o runtime recusar o tipo errado. Em Python, `package_count: int` é só uma **dica** para você e para o editor: nada impede passar `"abc"`. Por isso o **service** do projeto valida tudo manualmente.

4. Passeio pelos arquivos

4.1 database.py: a conexão e a criação da tabela

Laravel	Python (database.py)

`// config/database.php + .env`
`'sqlite' => [`
`  'driver' => 'sqlite',`
`  'database' => 'filafacil.db',`
`],`
`// Model: Illuminate\...\Model`
`class Pickup extends Model {}`
`// no terminal:`
`php artisan migrate`
	`DATABASE_URL = "sqlite:///filafacil.db"`
	`engine = create_engine(DATABASE_URL,`
	`  echo=False)`
	`SessionLocal = sessionmaker(`
	`  bind=engine, expire_on_commit=False)`
	
	`class Base(DeclarativeBase):`
	`  ...  # classe base dos models`
	
	`def create_tables() -> None:`
	`  import models.pickup`
	`  Base.metadata.create_all(engine)`

Conceito Python	O que é	Parecido com (Laravel)
`engine`	Objeto que sabe **como falar com o banco** (qual arquivo, qual driver).	A conexão configurada em config/database.php
`SessionLocal`	Fábrica de sessões. Cada `SessionLocal()` é uma conversa com o banco, com transação.	Abrir uma conexão / escopo de transação por operação
`Base (DeclarativeBase)`	Classe-mãe de todos os models. Guarda o registro das tabelas.	`Illuminate\Database\Eloquent\Model`
`create_all(engine)`	Cria as tabelas que **ainda não existem**.	`php artisan migrate`, mas sem histórico (veja o aviso abaixo)
`echo=False`	Se for `True`, imprime cada SQL gerado no terminal.	`DB::listen()` / query log / Telescope

> **create_all NÃO é uma migration de verdade**
> Ele só cria tabelas inexistentes. Se você adicionar uma coluna ao model e a tabela já existir, **nada muda no banco**. No Laravel você criaria uma nova migration. Aqui, como o projeto é de estudo, a solução é **apagar o filafacil.db** (equivale a `migrate:fresh`). O equivalente profissional das migrations no mundo SQLAlchemy se chama **Alembic** (não usado neste projeto).

4.2 models/pickup.py: o Model e a migration juntos

No Laravel você descreve as colunas na **migration** e o Model quase não tem colunas. No SQLAlchemy o **Model é a descrição da tabela**: cada coluna é declarada na classe, e o `create_all` gera o CREATE TABLE a partir dela.

Laravel (migration + model)	Python (models/pickup.py)

`Schema::create('pickups', function ($t) {`
`  $t->id();`
`  $t->string('customer_name', 100);`
`  $t->string('address', 200);`
`  $t->unsignedInteger('package_count');`
`  $t->string('priority', 10)`
`  ->default('medium');`
`  $t->string('status', 20)`
`  ->default('pending');`
`  $t->timestamp('created_at')`
`  ->useCurrent();`
`});`
	`class Pickup(Base):`
	`  __tablename__ = "pickups"`
	`  id: Mapped[int] = mapped_column(`
	`  Integer, primary_key=True,`
	`  autoincrement=True)`
	`  customer_name: Mapped[str] = mapped_column(`
	`  String(100), nullable=False)`
	`  package_count: Mapped[int] = mapped_column(`
	`  Integer, nullable=False)`
	`  priority: Mapped[str] = mapped_column(`
	`  String(10), nullable=False,`
	`  default="medium")`
	`  created_at: Mapped[datetime] = mapped_column(`
	`  DateTime, nullable=False,`
	`  default=datetime.now)`

- `Mapped[int]` diz o tipo Python do atributo; `mapped_column(Integer, ...)` diz o tipo da coluna no banco. Juntos fazem o papel dos **casts** do Eloquent + o tipo da migration.

- `nullable=False` equivale a **não** usar `->nullable()` na migration (coluna obrigatória).
- `default="medium"` é aplicado pelo SQLAlchemy no Python na hora do insert, como o `->default()`. Já `default=datetime.now` (sem parênteses!) passa a **função**, que roda a cada registro, como `useCurrent()`.
- `PRIORITIES` e `STATUSES` (tuplas) cumprem o papel de um **enum PHP 8.1**; `PRIORITY_LABELS` e `STATUS_LABELS` traduzem para o menu (como arquivos de tradução `lang/pt_BR`).
- No fim do arquivo há 3 `CheckConstraint`: o banco recusa `package_count <= 0` e valores fora das listas, mesmo que alguém insira por fora da aplicação. No Laravel isso seria um `DB::statement('ALTER TABLE ... ADD CHECK ...')` na migration (o Schema Builder não tem método nativo para isso).

4.3 repository/pickup_repository.py: o único que fala com o banco

Aqui mora a diferença conceitual mais importante do guia. Compare operação por operação:

Operação	Laravel (Eloquent)	Python (SQLAlchemy)
Buscar por id	`Pickup::find($id)`	`session.get(Pickup, pickup_id)`

Filtrar por campo	`Pickup::where('status', $s)`
	`->orderBy('id')->get()`
		`session.scalars(`
		`select(Pickup)`
		`.where(Pickup.status == s)`
		`.order_by(Pickup.id)).all()`

Busca parcial	`->where('customer_name', 'like', "%$t%")`	`.where(Pickup.customer_name` `.ilike(f"%{t}%"))`
Mais recente primeiro	`->orderByDesc('created_at')`	`.order_by(Pickup.created_at.desc())`

Criar	`Pickup::create([...])`	`session.add(pickup)`
		`session.commit()`
		`session.refresh(pickup)`

Atualizar	`$p->status = $s;` `$p->save();`	`pickup.status = s` `session.commit()`
Deletar	`$p->delete();`	`session.delete(pickup)` `session.commit()`

Active Record x Data Mapper (a grande diferença)

- **Eloquent é Active Record:** o próprio objeto sabe se salvar. `$pickup->save()` grava na hora.
- **SQLAlchemy ORM é Data Mapper com Unit of Work:** o objeto é só dados. Quem controla tudo é a **Session**: você faz `add`, `delete` ou altera atributos, e **nada vai ao banco até o** `commit()`. A Session é como uma transação aberta que acompanha os objetos.
- Por isso o repository sempre segue o ritmo: abre a sessão com `with`, mexe nos objetos, faz `commit()`. Se esquecer o commit, **os dados somem** ao fechar o programa.

> **expire_on_commit=False, o que é isso?**
> Por padrão, depois do `commit()` o SQLAlchemy "expira" os objetos: ao acessar um atributo depois que a sessão fechou, dá `DetachedInstanceError`. Como o repository devolve objetos para o service e o controller usarem **depois** de fechar a sessão, desligamos isso. É como garantir que o Model Eloquent continue com seus atributos carregados depois que a operação termina.

4.4 service/pickup_service.py: validação e regras

Laravel	Python (service)

`// FormRequest::rules()`
`'customer_name' => 'required|max:100',`
`'package_count' => 'required|integer|min:1',`
`'priority' => 'in:low,medium,high',`
 	
`// não achou o registro:`
`Pickup::findOrFail($id);`
`// -> ModelNotFoundException`
	`customer_name = customer_name.strip()`
	`if not customer_name:`
	`  raise ValueError(`
	`  "O nome do cliente é obrigatório.")`
	`if package_count <= 0:`
	`  raise ValueError("... maior que 0.")`
	`if priority not in PRIORITIES:`
	`  raise ValueError("Prioridade inválida.")`
	
	`# não achou o registro:`
	`raise LookupError(`
	`  f"Coleta #{pickup_id} não encontrada")`

Exceção Python	Quando é usada aqui	Parecido com (Laravel)
`ValueError`	Dado inválido: vazio, número fora do limite, valor fora da lista.	`ValidationException`
`LookupError`	A coleta pedida não existe (atualizar ou deletar um id inexistente).	`ModelNotFoundException` (do `findOrFail`)

Note a ordem em `update_status` e `delete`: o service chama `get_by_id` **antes** de agir, garantindo a existência (requisito do case). Em Laravel isso seria o `findOrFail` no começo do método.

4.5 controller/pickup_controller.py: o menu

Como dito na seção 1, pense num **Artisan Command interativo**. Cada método do controller faz o que um comando faria:

Controller do projeto	Artisan Command (Laravel)
`input("Nome do cliente: ")`	`$this->ask('Nome do cliente')`
`choose_option(...)` (lista numerada de prioridades)	`$this->choice('Prioridade', ['Baixa', 'Média', 'Alta'])`
`print_table(pickups)`	`$this->table($headers, $rows)`
`print("Coleta #1 cadastrada.")`	`$this->info('Coleta #1 cadastrada.')`
`print(f"Erro: {error}")`	`$this->error($e->getMessage())`
`while True:` + `input()` (o menu em loop)	Um comando com `while (true)` e `$this->choice()`

- O controller **nunca importa SQLAlchemy nem o repository**: só chama o service. É a regra que impede "query dentro do controller".
- Ele converte a entrada (`int(texto)`) e captura `ValueError` / `LookupError` para mostrar mensagens em português em vez de um traceback (o equivalente à tela de erro 500 do Laravel).
- Os rótulos em português (`PRIORITY_LABELS`) ficam só aqui na exibição: o banco sempre guarda os valores em inglês (`high`, `in_transit`).

4.6 main.py: ligando tudo

Laravel (Service Container)	Python (main.py)

`// o container resolve sozinho:`
`public function __construct(`
`  private PickupService $service`
`) {}`
 	
`// Laravel descobre PickupService,`
`// depois PickupRepository, etc.`
	`def main() -> None:`
	`  create_tables()`
	`  controller = PickupController(`
	`  PickupService(`
	`  PickupRepository()))`
	`  controller.run()`
	
	`if __name__ == "__main__":`
	`  try:`
	`  main()`
	`  except (KeyboardInterrupt, EOFError):`
	`  print("Encerrado pelo usuário.")`

- **Injeção de dependência manual:** Python não tem service container. Quem monta `PickupController(PickupService(PickupRepository()))` é o `main.py`, e cada classe recebe a dependência no `__init__` (como no `__construct`).
- `if __name__ == "__main__":` significa "só execute isto se o arquivo foi rodado diretamente (`python main.py`) e não apenas importado". É o papel do arquivo `artisan` como ponto de entrada.
- O `try/except` em volta captura o **Ctrl+C** para encerrar com mensagem amigável, sem mostrar o traceback.

5. Comandos do dia a dia

Laravel / PHP	Este projeto / Python
`composer install`	`pip install -r requirements.txt` (ou `uv pip install -r ...`)
`composer require pacote`	`pip install pacote` e anotar no `requirements.txt` (o pip não grava sozinho)
(Composer já instala por projeto)	`python3 -m venv .venv` e depois `source .venv/bin/activate`. O ambiente virtual isola as libs do projeto.
`php -v` / versão no composer.json	`python --version`; o `.python-version` fixa a versão do projeto (3.11)
`php artisan serve` (rodar o app)	`python main.py`
`php artisan migrate`	Automático: o `main.py` chama `create_tables()` em toda execução
`php artisan migrate:fresh`	`rm filafacil.db`, depois rodar de novo
`php artisan tinker`	`python` (REPL) ou `python -i arquivo.py`
`vendor/autoload.php` (PSR-4)	Imports a partir da pasta do projeto. Por isso se roda **de dentro da raiz**: `from models.pickup import Pickup`
PHPUnit / Pest	pytest (não usado neste projeto: os testes foram manuais e por scripts)
Ver o banco: TablePlus / DBeaver	Extensão SQLite Viewer no editor, ou `sqlite3 filafacil.db`

Como rodar este projeto do zero

```
cd projects/junior/simple-database-manegement
python3.11 -m venv .venv          # cria o "vendor" do Python
source .venv/bin/activate         # ativa o ambiente
pip install -r requirements.txt   # composer install
python main.py                    # php artisan serve (aqui: abre o menu)
```

Dica de aprendizado
Para ver exatamente o que o SQLAlchemy envia ao banco, mude `echo=False` para `echo=True` em `database.py` e use o menu. Você verá cada INSERT, SELECT, UPDATE e DELETE no terminal, como um `DB::listen()` no Laravel.

6. Armadilhas para quem vem do Laravel

#	Armadilha	Explicação
1	Indentação é sintaxe	Misturar tabs e espaços ou errar um nível quebra o programa ou muda o que o código faz. Use sempre 4 espaços.
2	Sem "mágica"	Não há facades, helpers globais nem container automático. Tudo vem de `import` explícito e de construtores. Mais verboso, mais fácil de rastrear.
3	Ambiente virtual	No Laravel as dependências ficam em vendor/ automaticamente. Em Python é preciso criar e **ativar** o `.venv`; sem ativar, o `python` do sistema não enxerga o SQLAlchemy.
4	Falsy diferente	Em PHP, a string `"0"` é falsa. Em Python, só `""`, `0`, `None` e coleções vazias são falsas: `"0"` é verdadeira.
5	Strings são imutáveis	`name.strip()` **devolve** um texto novo, não altera o original. Por isso o service faz `name = name.strip()`.
6	Inteiros sem limite	Python aceita números gigantes (em PHP estouraria para float). O SQLite só guarda 64 bits: um pacote de 10^30 causou `OverflowError` neste projeto. Hoje o service limita os valores.
7	SQLite não impõe tamanho	`String(100)` **não impede** 500 caracteres no SQLite (diferente de MySQL em modo estrito). Por isso o service valida o tamanho. Já as `CHECK` constraints o SQLite respeita.
8	Importação circular	O model importa `database` (para pegar o `Base`) e o `database` precisa conhecer o model. Solução usada: importar `models.pickup` **dentro** de `create_tables()`.
9	`is None` e não `== None`	Para comparar com None use `is None`. Para comparar valores use `==`. É o equivalente a `=== null`.
10	Esquecer o commit	Sem `session.commit()` nada é gravado. No Eloquent o `save()` já grava.

7. Glossário rápido

Termo	Significado simples
ORM	Mapeia tabelas para classes. Eloquent no Laravel; **SQLAlchemy** aqui.
Engine	A conexão configurada com o banco.
Session	Uma conversa com o banco que acompanha os objetos e controla a transação. Nada é gravado até o `commit()`.
DeclarativeBase / Base	Classe-mãe dos models (o `Model` do Eloquent).
Mapped / mapped_column	Forma moderna (SQLAlchemy 2.x) de declarar colunas com tipo Python e tipo de banco.
Context manager (with)	Bloco que abre um recurso e o fecha sozinho no fim, mesmo com erro.
Type hint	Anotação de tipo (`x: int`). Documenta e ajuda o editor; não é validada em execução.
Docstring	Texto entre aspas triplas logo abaixo de `def`/`class`, que documenta o código (PHPDoc).
venv / .venv	Pasta com o Python e as libs só deste projeto.
requirements.txt	Lista das dependências (`composer.json`).
Repository / Service / Controller	Camadas: acesso a dados / regras de negócio / interação com o usuário.
CHECK constraint	Regra gravada na própria tabela que o banco faz cumprir (ex.: pacotes maior que 0).
Active Record x Data Mapper	Objeto que se salva sozinho (Eloquent) x objeto simples salvo por um gerenciador, a Session (SQLAlchemy).

8. Para praticar (sugestões em ordem de dificuldade)

1. **Ver o SQL:** troque `echo=False` por `True` e acompanhe cada operação do menu. Compare com o que o Eloquent geraria.
2. **Olhar o banco:** rode `sqlite3 filafacil.db ".schema"` e confira as 7 colunas e as 3 constraints criadas a partir do model.
3. **Sentir o limite do create_all:** adicione uma coluna `phone` ao model, rode o programa e veja que ela não aparece. Depois apague o `filafacil.db` e rode de novo. Pense em como isso seria uma nova migration no Laravel.
4. **Quebrar de propósito:** no `service`, comente a validação de pacotes e tente cadastrar 0. Veja a constraint do banco recusar. Depois descomente.
5. **Ler no sentido contrário:** escolha uma opção do menu e siga a chamada de arquivo em arquivo (controller, service, repository) até o SQL. É o mesmo exercício de seguir uma rota até o banco no Laravel.

> **Fecho**
> Se você já entende Controller, Service, Repository e Model no Laravel, **você já entende a arquitetura deste projeto**. O que muda é a sintaxe do Python e o modelo do SQLAlchemy (Session em vez de Active Record). Foque nesses dois pontos.